Generative AI Analysis and Ethics Report

Recipe Generation Using Fine-Tuned GPT-2

Faheema Mohamed Maideen | Generative AI Capstone Project

1. Overview

This report describes the design, implementation, and evaluation of an ingredient-conditioned recipe generation system. The system takes a comma-separated list of cooking ingredients as input and generates a complete recipe, including a title and step-by-step cooking directions. The chosen generative approach is Transformer-based text generation, specifically fine-tuning a pretrained GPT-2 model (Radford et al., 2019) on the RecipeNLG dataset (Bień et al., 2020). GPT-2 is a decoder-only Transformer architecture that uses autoregressive language modeling to generate coherent multi-sentence text. This task is well-suited to a Transformer-based approach because recipe generation requires domain-specific vocabulary, logical step ordering, and coherent multi-sentence output — capabilities that benefit from pretrained language understanding rather than training from scratch.

2. Dataset or Prompt Description

The dataset used in this project is RecipeNLG (Bień et al., 2020), a large-scale cooking recipe dataset containing 2,231,142 recipes scraped from publicly available cooking websites. Each recipe entry includes a title, a list of raw ingredients with measurements, step-by-step directions, named entity recognition (NER) tags for clean ingredient names, and a source URL. The dataset is publicly available on Kaggle for non-commercial research and educational use, and is not synthetic or AI-generated.

For this project, recipes were filtered to the “Gathered” source category (511,261 recipes) to ensure consistent formatting quality. A random sample of 50,000 recipes was selected for fine-tuning to balance training feasibility with dataset coverage. The NER column was used as the input condition (clean ingredient names without quantities), and the full title and directions were used as the generation target. Each recipe was formatted as a structured string using special tokens:

```
<|startofrecipe|>
Ingredients: [NER ingredient list]
Title: [recipe title]
```

```
Directions: [step-by-step directions]
<|endofrecipe|>
```

Text length analysis confirmed that approximately 98% of recipes fit within 512 tokens, making this the appropriate maximum sequence length for tokenization.

3. Model Design and Training Approach

The generative model is GPT-2 Small (Radford et al., 2019), a decoder-only Transformer with 124 million parameters pretrained on WebText, a large English text corpus. GPT-2 was selected over training a Transformer from scratch because fine-tuning leverages pretrained language knowledge, which is critical for generating coherent cooking terminology and logical step sequences (Howard & Ruder, 2018).

The tokenizer was extended with three special tokens: `<|pad|>`, `<|startofrecipe|>`, and `<|endofrecipe|>`, expanding the vocabulary from 50,257 to 50,260 tokens. Token embeddings were resized accordingly.

Key Design Decisions

- **Causal language modeling objective:** The model predicts each next token given all previous tokens (cross-entropy loss). Padding positions were masked with -100 to exclude them from the loss computation.
- **AdamW optimizer (lr = 5e-5, weight_decay = 0.01):** The standard optimizer for Transformer fine-tuning. AdamW applies weight decay correctly, decoupled from gradient updates, helping prevent overfitting (Loshchilov & Hutter, 2019).
- **Linear warmup + decay schedule:** 10% warmup steps followed by linear decay, stabilizing early training and preventing large gradient updates before the model has adapted to the recipe domain.
- **Gradient accumulation (4 steps):** Achieves an effective batch size of 16 while keeping per-step memory usage low (physical batch size = 4).
- **Gradient clipping (max_norm = 1.0):** Prevents exploding gradients during fine-tuning.
- **Mixed precision (torch.autocast, float16):** Used on CUDA to reduce memory usage and speed up training.
- **Early stopping (patience = 3):** Monitors validation loss and restores the best checkpoint.

Training ran for 2 epochs on 45,000 recipes (90/10 train/val split), with the best model checkpoint saved based on validation loss. Final training loss: 1.9284 (perplexity: 6.88). Final validation loss: 1.7745 (perplexity: 5.90). The lower validation loss relative to training loss indicates stable generalization without overfitting.

Text generation used top-p nucleus sampling (p = 0.9), top-k sampling (k = 50), temperature = 0.8, and no-repeat-ngram-size = 3 to balance output diversity and coherence (Holtzman et al., 2020).

4. Output Evaluation and Interpretation

Twenty recipes were generated using diverse ingredient sets to evaluate model performance across different cuisine types and complexity levels.

Strengths

The model consistently produced well-structured outputs following the trained format. In most cases, the generated title was relevant to the input ingredients. For example, given “cream cheese, sugar, vanilla, graham cracker, eggs,” the model generated “Grandma’s Cream Cheese Cake” with a logical baking sequence. Given “rice, black beans, corn, salsa, cheese,” the model generated “Mexican Rice” with multi-step directions that correctly used the provided ingredients. Quantitatively, the validation perplexity of 5.90 indicates the model assigns high probability to coherent recipe text, reflecting meaningful adaptation to the domain.

Failure Cases

Several systematic failure patterns were observed:

1. **Vague or incomplete directions:** Given “beef, onion, potato, carrot, broth,” the model generated “Beef Stew” with only a single vague step — “Mix all ingredients together and bake in oven” — ignoring the broth entirely. This reflects the model’s tendency to collapse complex dishes to the nearest high-frequency training pattern.
2. **Hallucinated steps:** Recipe #6 (shrimp, garlic, butter, lemon, parsley) included the step “simmer until parsley melts,” which is culinarily nonsensical. Recipe #2 (chocolate cake) instructed the user to “pour in 1 cup cold water” mid-bake — a step not grounded in the input or realistic baking practice.
3. **Ingredient count hallucination:** Recipe #7 (banana, peanut butter, oats, honey — 4 ingredients) referenced “Mix first 5 ingredients,” hallucinating a fifth ingredient that was not provided.
4. **Thematic mismatch:** Given “sweet potato, black bean, cumin, lime, cilantro,” the model generated “Hot Vegetable Soup” rather than a more contextually appropriate dish such as a burrito bowl or taco filling.

These failure modes are consistent with documented limitations of autoregressive language models fine-tuned on limited data (Holtzman et al., 2020).

5. Ethical Considerations and Responsible Use

Bias in Training Data

The RecipeNLG dataset was scraped predominantly from English-language Western cooking websites. As a result, the model demonstrates a strong bias toward American and European recipe styles and cooking methods (e.g., oven baking at 350°F appears in nearly every generated recipe, regardless of cuisine type). Ingredient combinations associated with East Asian, South Asian, or Latin American cuisines produced outputs that either mapped to generic Western formats or generated thematically mismatched titles. For example, “tofu, broccoli, soy sauce, sesame oil, rice” generated a recipe that ended with “Bake at 350° for 1 hour” — a method inconsistent with typical stir-fry preparation. This reflects a systemic bias in the training corpus that could lead users from underrepresented culinary traditions to receive inaccurate or culturally inappropriate recipe guidance.

Creative Ownership

The model generates text that is statistically derived from millions of scraped recipes created by human cooks and food writers. The generated recipes are not attributed to any source, raising questions about intellectual property and the uncredited use of human creative work as training data (Bender et al., 2021). Users or developers deploying such a system commercially should consider data provenance, licensing, and compensation for original content creators.

Misuse Potential

While a recipe generator poses limited direct harm, the same fine-tuning methodology applied to sensitive domains (medical instructions, legal advice, financial guidance) could produce plausible-sounding but incorrect outputs that users may trust inappropriately. This project illustrates the general risk of small fine-tuned language models generating confidently-stated but factually incorrect content — a form of hallucination that extends beyond recipe generation.

6. Limitations and Future Improvements

Limitations

- Training on 50,000 recipes (2.2% of the full dataset) limits the model’s exposure to diverse recipe patterns, contributing to repetitive outputs and over-reliance on common structures.
- GPT-2 Small (124M parameters) has limited capacity compared to modern large language models, restricting its ability to generate nuanced multi-step directions.
- The model does not verify ingredient consistency: it may generate directions using ingredients not in the input list, or omit provided ingredients entirely.
- No quantitative evaluation metric beyond perplexity was applied; metrics such as BLEU or ingredient coverage rate would provide more targeted quality measurement.

- The dataset's Western bias limits usefulness for global culinary applications.

Future Improvements

- Fine-tune on the full 2.2M-recipe dataset to improve coverage and reduce repetition.
- Use GPT-2 Medium or Large (345M–774M parameters) to improve output coherence.
- Add an ingredient coverage scoring step that checks whether all input ingredients appear in the generated output.
- Introduce cuisine-type conditioning (e.g., "Cuisine: Italian") to improve thematic alignment.
- Evaluate outputs using BLEU scores against held-out reference recipes and human evaluation rubrics.

7. References

- Bień, M., Gilski, M., Maciejewska, M., Taisner, W., Wisniewski, D., & Lawrynowicz, A. (2020). RecipeNLG: A cooking recipes dataset for semi-structured text generation. In *Proceedings of the 13th International Conference on Natural Language Generation (INLG)*, 22–28. Association for Computational Linguistics.
- Bender, E. M., Gebru, T., McMillan-Major, A., & Shmitchell, S. (2021). On the dangers of stochastic parrots: Can language models be too big? In *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, 610–623. https://doi.org/10.1145/3442188.3445922
- Holtzman, A., Buys, J., Du, L., Forbes, M., & Choi, Y. (2020). The curious case of neural text degeneration. In *Proceedings of the International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/1904.09751
- Howard, J., & Ruder, S. (2018). Universal language model fine-tuning for text classification. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, 328–339. https://doi.org/10.18653/v1/P18-1031
- Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. In *Proceedings of the International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/1711.05101
- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). Language models are unsupervised multitask learners. *OpenAI Blog, 1*(8), 9. https://openai.com/research/language-unsupervised